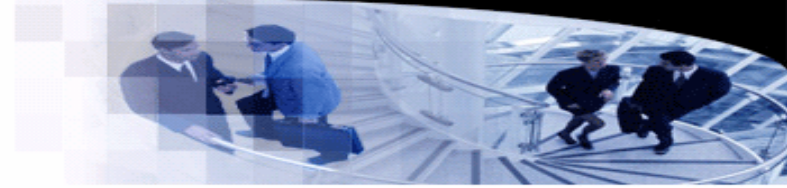




武汉大学
WUHAN UNIVERSITY



Programming Languages and Compilers

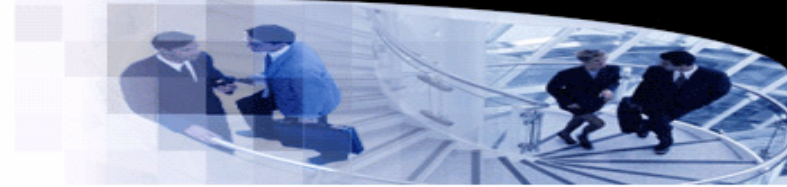
Lecture 8. Lambda-Calculus

袁梦霆 (YUAN Mengting)

ymt@whu.edu.cn

School of Computer Science, Wuhan University

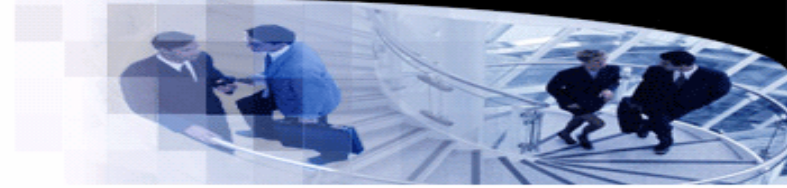
Lambda-Calculus



- Brief introduction

- A formal system invented in the 1920s by Alonzo Church, in which all computation is reduced to the basic operations of function definition and application.
- Nameless functions are used to represent functional abstractions.
- Computations are formalized as function applications.
- It is a popular core language for many formal languages.
- Many modern programming languages ship with lambda calculus.

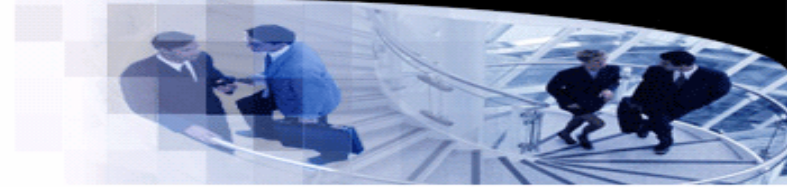
Lambda-Calculus



- Basics

- Functional abstractions are used to avoid writing similar calculations.
- Consider the expression $(5 * 4 * 3 * 2 * 1) + (3 * 2 * 1) + (2 * 1)$. It contains three similar terms.
- If we define a function `factorial(n) = if n = 0 then 1 else n * factorial(n - 1)`, then the expression can be written in `factorial(5) + factorial(3) + factorial(2)`.
- The function can be defined in a lambda-calculus style.
 - `factorial = λn. if n = 0 then 1 else n * factorial (n - 1)`
- `factorial (0)` means “the function `(λn. if n=0 then 1 else ...)` applied to the argument 0,” that is “the value that results when the argument variable `n` in the function body `(λn. if n=0 then 1 else ...)` is replaced by 0,” that is “if `n=0 then 1 else ...`,” that is, 1.

Lambda-Calculus



- **Syntax**

- $t ::=$
- x
- $\lambda x. t$
- $t \ t$

terms:
variable
abstraction
application

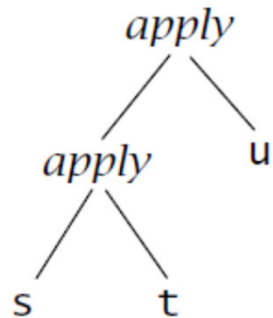
- **Examples:**

- $x \ y$: y is applied to term x .
- $\lambda x. x$: a function that returns the argument x it self.
- $\lambda x. \lambda y. x \ y$: a function that takes an argument x and returns $\lambda y. x \ y$, which takes an argument y and returns a term $x \ y$.

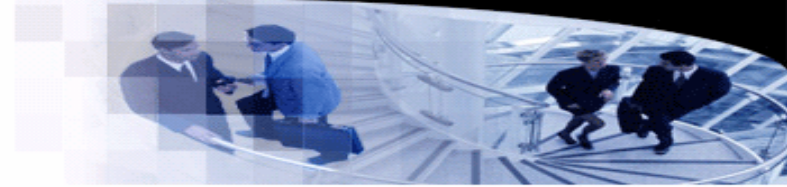
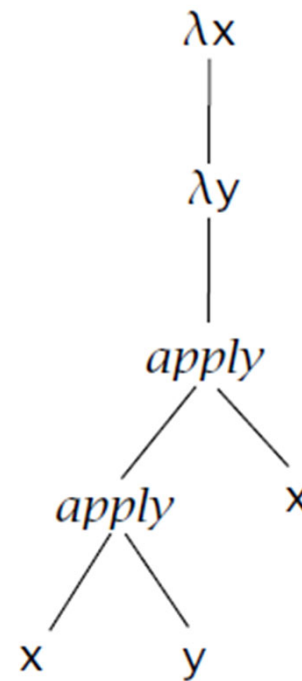
Lambda-Calculus

- Syntax trees

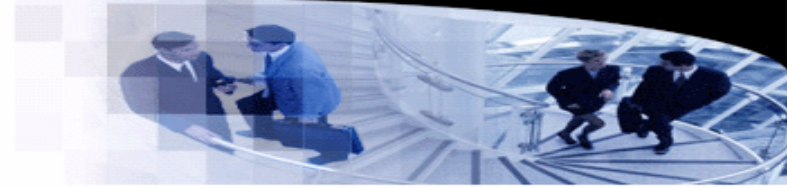
- $(s\ t)\ u$



- $\lambda x.\lambda y.x\ y\ x = \lambda x.(\lambda y.((x\ y)\ x))$



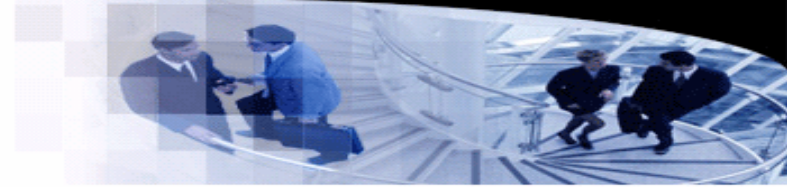
Lambda-Calculus



- **Semantics**

- Lambda-calculus does not provide popular built-in programming features like constants, arithmetic operations, branches, or iterations.
- A computation of lambda-calculus is to apply an argument to an abstraction.
- The application replaces every bound variable in a term with the given argument term.
- $(\lambda x. t12) \ t2 \rightarrow [x \mapsto t2] t12$ means “replace every free x in $t12$ with $t2$ to generate a new term”.
- Examples:
 - $(\lambda x. x) \ y \rightarrow y$
 - $(\lambda x. x \ (\lambda x. x)) \ (u \ r) \rightarrow u \ r \ (\lambda x. x)$

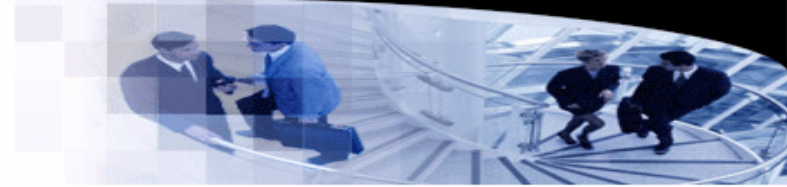
Lambda-Calculus



- **Semantics**

- A term $(\lambda x. t_1) t_2$ is called a **redex** (reducible expression).
- The operation of rewriting a redex according to the substitution rule is called **beta-reduction**.
- There are several different evaluation strategies for lambda-calculus.
- Each strategy defines which redex or redexes in a term can fire on the next step of evaluation.

Lambda-Calculus



- **Semantics**

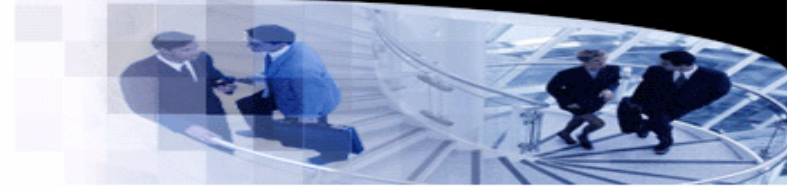
- **Full beta-reduction:** any redex may be reduced at any time.
- Consider the term $(\lambda x. x) ((\lambda x. x) (\lambda z. (\lambda x. x) z))$. It can be written in `id (id (λz. id z))`.
- It has three redexs.

$$\frac{\text{id (id (λz. id z))}}{\text{id ((id (λz. id z)))}} \\ \text{id (id (λz. id z))}$$

- Under full beta-reduction, we we might choose to begin with the innermost redex, then do the one in the middle, then the outermost.

$$\begin{aligned} & \text{id (id (λz. id z))} \\ \rightarrow & \text{id (id (λz. z))} \\ \rightarrow & \text{id (λz. z)} \\ \rightarrow & \lambda z. z \\ \not\rightarrow & \end{aligned}$$

Lambda-Calculus



- **Semantics**

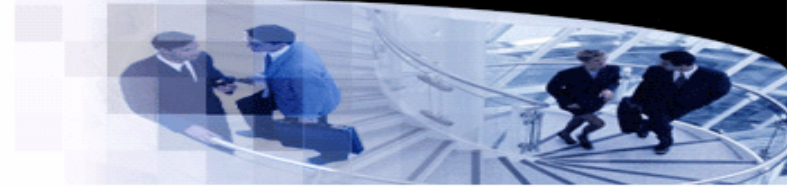
- **Normal order strategy:** the leftmost, outermost redex is always reduced first.

$$\begin{aligned} & \text{id (id (\lambda z. id z))} \\ \rightarrow & \text{id (\lambda z. id z)} \\ \rightarrow & \lambda z. \text{id z} \\ \rightarrow & \lambda z. z \\ \not\rightarrow & \end{aligned}$$

- **Call by name strategy:** allowing no reductions inside abstractions. Variants of call by name have been used in some well-known programming languages like Haskell (call by need).

$$\begin{aligned} & \text{id (id (\lambda z. id z))} \\ \rightarrow & \text{id (\lambda z. id z)} \\ \rightarrow & \lambda z. \text{id z} \\ \not\rightarrow & \end{aligned}$$

Lambda-Calculus



- **Semantics**

- **Call by value strategy:** only outermost redexes are reduced and where a redex is reduced only when its right-hand side has already been reduced to a value—a term that is finished computing and cannot be reduced any further.

$$\begin{aligned} & \text{id } (\text{id } (\lambda z. \text{id } z)) \\ \rightarrow & \text{id } (\lambda z. \text{id } z) \\ \rightarrow & \lambda z. \text{id } z \\ \not\rightarrow & \end{aligned}$$

- Most programming languages use the call by value strategy.
- The call-by-value strategy is strict, in the sense that the arguments to functions are always evaluated, whether or not they are used by the body of the function.
- In contrast, non-strict (or lazy) strategies such as call-by-name and call-by-need evaluate only the arguments that are actually used.